



ARTIFICIAL INTELLIGENCE

VI SEMESTER



UNIT-II

Logical Reasoning

- **Logical Agents**
- **Propositional Logic**
- **First Order Logic**
- **Inferences**
- **Forward and Backward Chaining**
- **Unification and Resolution**



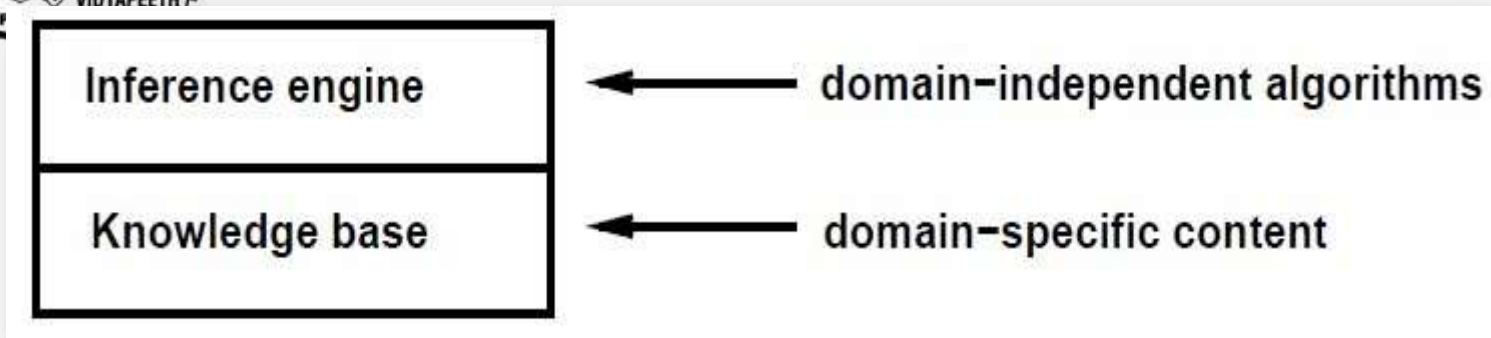
Why Do We Need Logic?

- Problem-solving agents were very inflexible: hard code every possible state.
- Search is almost always exponential in the number of states.
- Problem solving agents cannot infer unobserved information.
- We want an agent that can reason similarly to humans.





Knowledge Base



- **Knowledge base**: Set of sentences in a formal language
- Declarative approach for building an agent
 - **TELL** it what it needs to know
 - Then it can **ASK** itself what to do - answers should follow from the KB
- Agents can be viewed at the **knowledge level**
 - What they know, regardless of implementation
- Or at the **implementation level**
 - Data structures in KB and algorithms that manipulate them.

A Knowledge-based Agent

```
function KB-AGENT(percept) returns an action
  static: KB, a knowledge base
         t, a counter, initially 0, indicating time
  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  action ← ASK(KB, MAKE-ACTION-QUERY(t))
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t ← t + 1
  return action
```

- The agent must be able to:
 - Represent states, actions, etc.
 - Incorporate new percepts
 - Update internal representations of the world
 - Deduce hidden properties of the world
 - Deduce appropriate actions.



Wumpus World

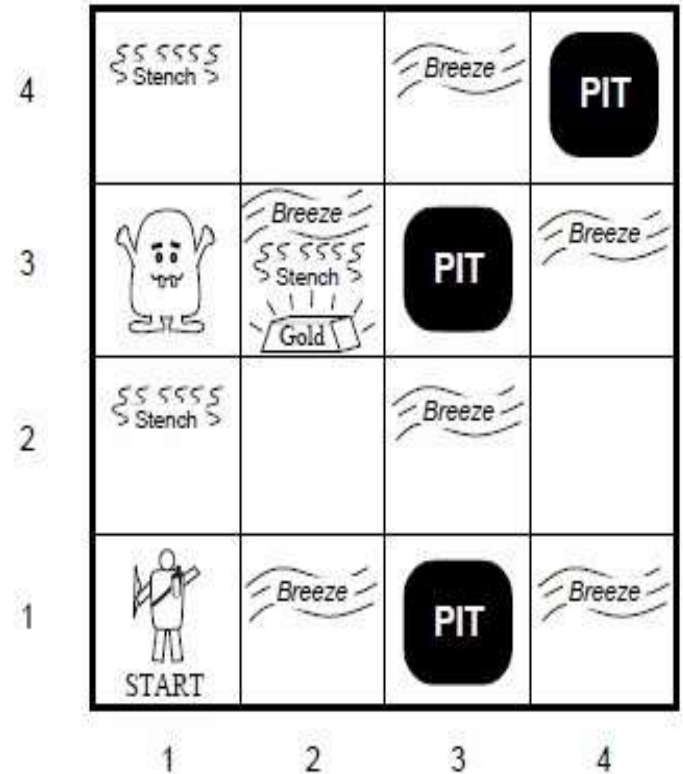
Performance measure: Gold +1000,
Death -1000, -1 per step, -10 for using the
arrow

Environment:

- Squares adjacent to wumpus are smelly
- Squares adjacent to pit are breezy
- Glitter iff gold is in the same square
- Shooting kills wumpus if you are facing it
- Shooting uses up the only arrow
- Grabbing picks up gold if in same square
- Releasing drops the gold in same square

Actuators: Left turn, Right turn, Forward,
Grab, Release, Shoot

Sensors: Breeze, Glitter, Smell.





Wumpus World Characterization

- **Observable?** No - only local perception
- **Deterministic?** Yes -outcomes exactly specified
- **Episodic?** No - sequential at the level of actions
- **Static?** Yes -Wumpus and Pits do not move
- **Discrete?** Yes
- **Single-agent?** Yes





Exploring the Wumpus World

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2	3,2	4,2
1,1 A OK	2,1 OK	3,1	4,1

- A** = Agent
- B** = Breeze
- G** = Glitter, Gold
- OK** = Safe square
- P** = Pit
- S** = Stench
- V** = Visited
- W** = Wumpus



Exploring the Wumpus World

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1



Exploring the Wumpus World

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

- A** = Agent
- B** = Breeze
- G** = Glitter, Gold
- OK** = Safe square
- P** = Pit
- S** = Stench
- V** = Visited
- W** = Wumpus



Exploring the Wumpus World

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

1,4	2,4 P?	3,4	4,4
1,3 W!	2,3 A S G B	3,3 P?	4,3
1,2 S V OK	2,2 V OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1



Logic In General

- Logics are formal languages for representing information, such that conclusions can be drawn
- Syntax defines the sentences in the language
- Semantics define the “meaning” of sentences, i.e., define **truth** of a sentence in a world
- E.g., the language of arithmetic
- $x + 2 \geq y$ is a sentence; $x^2 + y >$ is not a sentence
- $x + 2 \geq y$ is true iff the number $x + 2$ is no less than the number y
- $x + 2 \geq y$ is true in a **world** where $x=7$; $y=1$
- $x + 2 \geq y$ is false in a **world** where $x=0$; $y=6$



Entailment

- Entailment means that one thing **follows** from another

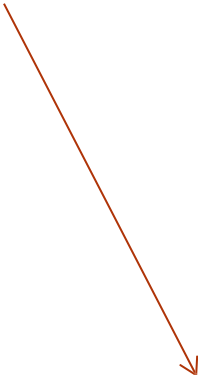
$$KB \models \alpha$$

- Knowledge base KB entails sentence B iff B is true in all worlds where KB is true
 - E.g., the KB containing “the Giants won” and “the Reds won” entails “Either the Giants won or the Reds won”
 - E.g., $x + y = 4$ entails $4 = x + y$
- Entailment is a relationship between sentences (i.e., syntax) that is based on semantics.



Models

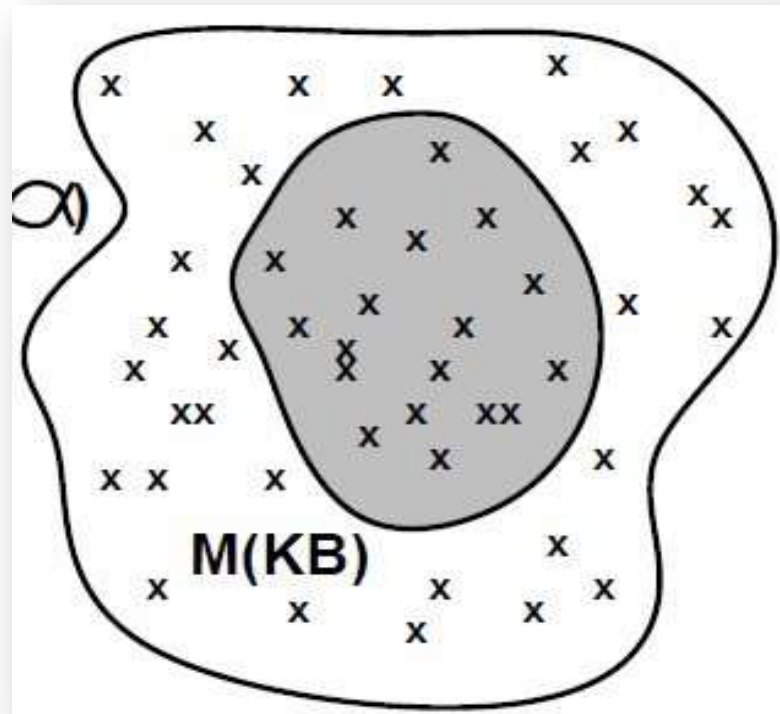
- Logicians typically think in terms of **models**, which are formally structured worlds with respect to which truth can be evaluated
- We say **m** is a model of a sentence B if B is true in **m**
 - $M(B)$ is the set of all models of B
- Then $KB \models B$ if and only if $M(KB) \subseteq M(B)$
 - E.g. $KB = \text{Giants won and Reds won}$
 - $B = \text{Giants won}$



$M(B)$ could be also true for worlds that are different than the worlds of KB

Models

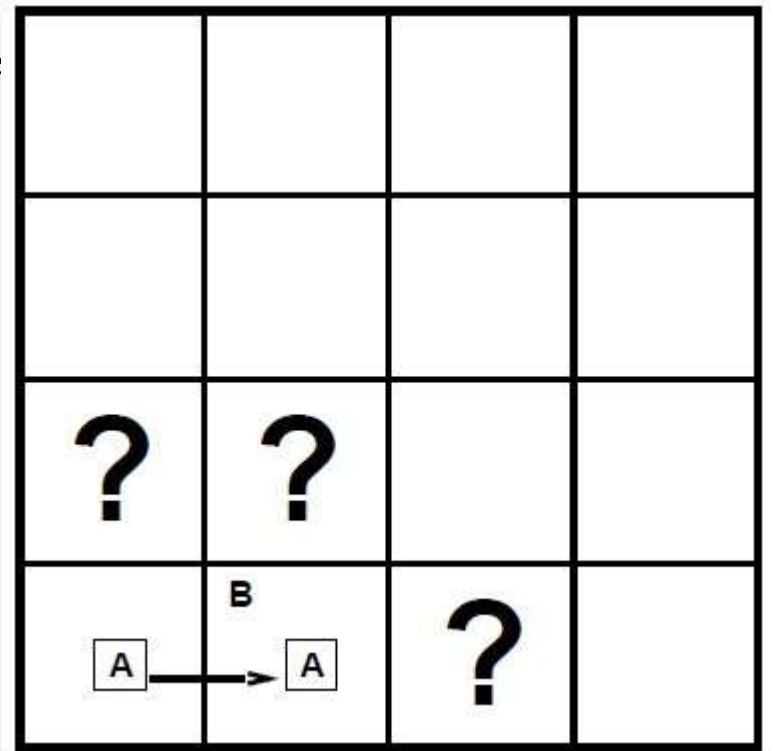
$M(B)$



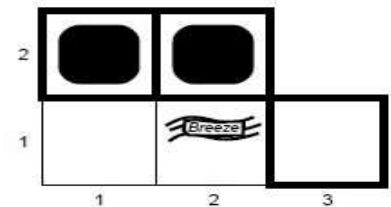
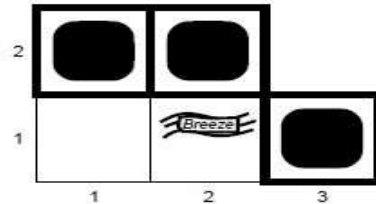
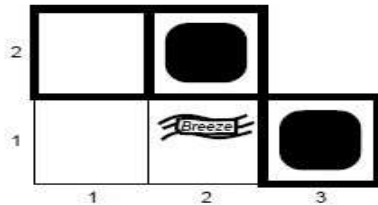
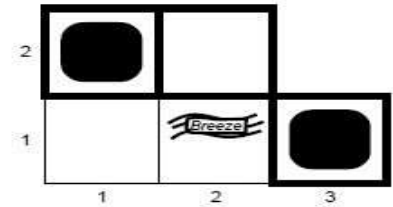
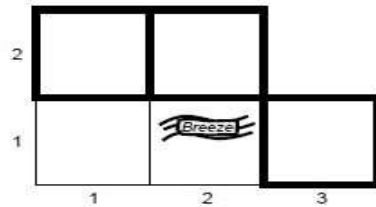
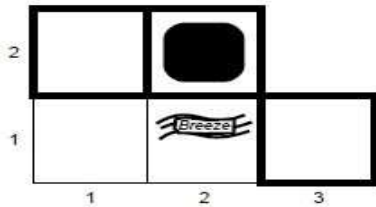
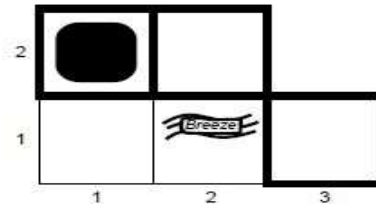
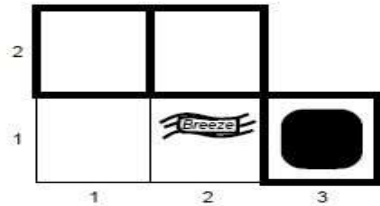


Entailment in the Wumpus World

- Situation after detecting nothing in $[1,1]$, moving right, breeze in $[2,1]$
- Consider possible models for ?s, assuming only pits
- 3 Boolean choices, i.e., 8 possible models

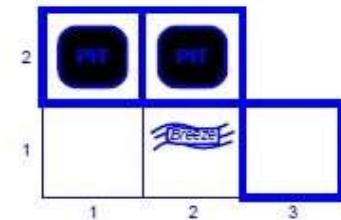
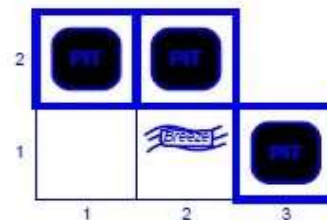
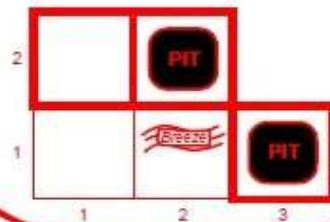
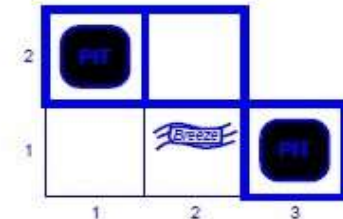
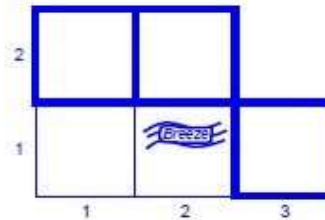
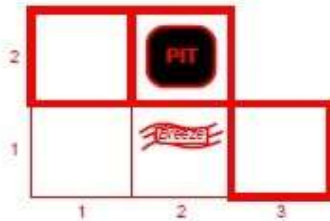
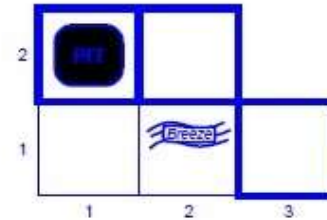
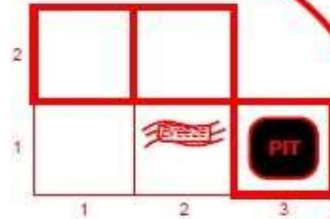


Wumpus Models



Wumpus Models

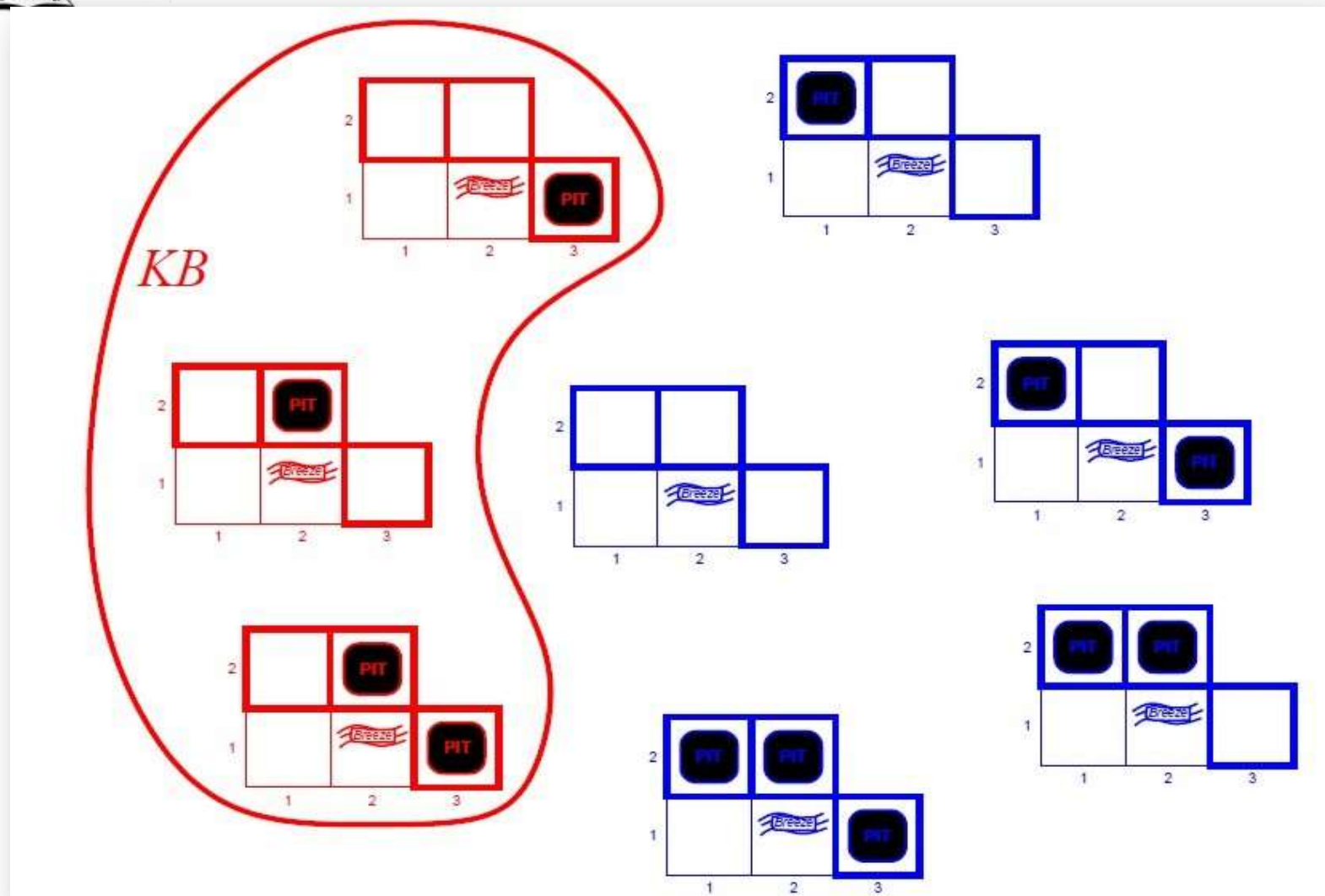
KB



KB = Wumpus World Rules + Observations

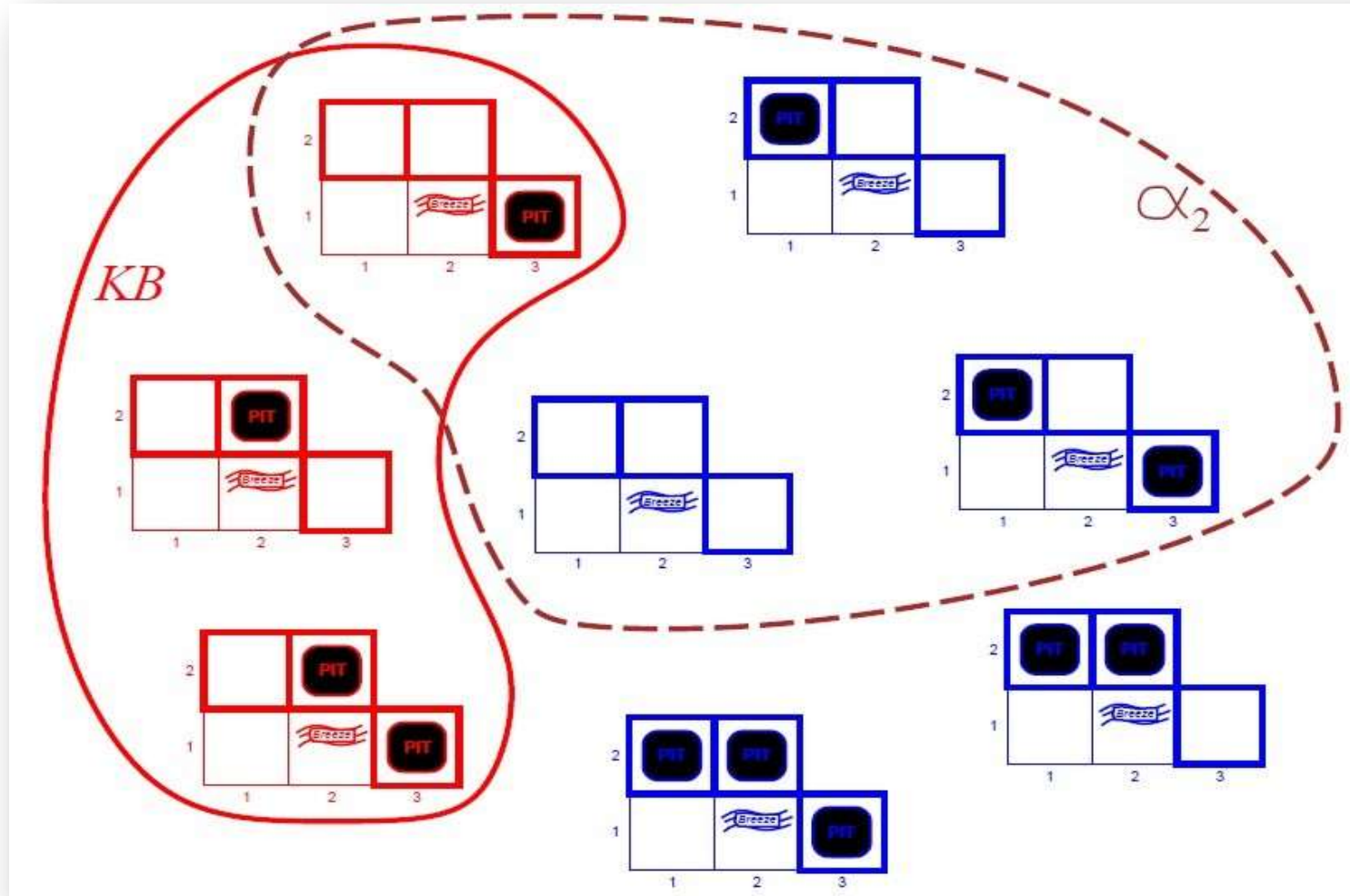


Wumpus Models



**KB = Wumpus World Rules +
Observations**

Wumpus Models



KB = Wumpus World Rules + Observations

Alpha_2 = "(2,2) is safe, KB $\models$ Alpha_2 is false"



Inference

$KB \vdash_i \alpha$ = sentence α can be derived from KB by procedure i

Consequences of KB are a haystack; α is a needle.

Entailment = needle in haystack; inference = finding it

Soundness: i is sound if

whenever $KB \vdash_i \alpha$, it is also true that $KB \models \alpha$

Completeness: i is complete if

whenever $KB \models \alpha$, it is also true that $KB \vdash_i \alpha$

- **First Order Logic (FOL):** Allows complete and sound inference procedures
 - Will answer any question whose answer follows from what is known by the KB.



Propositional Logic

Propositional logic is the simplest logic—illustrates basic ideas

The proposition symbols P_1, P_2 etc are sentences

If S is a sentence, $\neg S$ is a sentence (negation)

If S_1 and S_2 are sentences, $S_1 \wedge S_2$ is a sentence (conjunction)

If S_1 and S_2 are sentences, $S_1 \vee S_2$ is a sentence (disjunction)

If S_1 and S_2 are sentences, $S_1 \Rightarrow S_2$ is a sentence (implication)

If S_1 and S_2 are sentences, $S_1 \Leftrightarrow S_2$ is a sentence (biconditional)

Order of Precedence

If and Only If

If S_1 then S_2



Propositional Logic: Semantics

Each model specifies true/false for each proposition symbol

E.g. $P_{1,2}$ $P_{2,2}$ $P_{3,1}$

true true false If S_1 is true, then I am claiming that S_2 is true

(With these symbols, 8 possible models, can be enumerated automatically.)

Rules for evaluating truth with respect to a model m :

$\neg S$ is true iff	S is false
$S_1 \wedge S_2$ is true iff	S_1 is true and S_2 is true
$S_1 \vee S_2$ is true iff	S_1 is true or S_2 is true
$S_1 \Rightarrow S_2$ is true iff	S_1 is false or S_2 is true
i.e., is false iff	S_1 is true and S_2 is false
$S_1 \Leftrightarrow S_2$ is true iff	$S_1 \Rightarrow S_2$ is true and $S_2 \Rightarrow S_1$ is true

Simple recursive process evaluates an arbitrary sentence, e.g.,

$\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1}) = \text{true} \wedge (\text{false} \vee \text{true}) = \text{true} \wedge \text{true} = \text{true}$



Truth Table for Connectives

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true



Wumpus World Sentences

Let $P_{i,j}$ be true if there is a pit in $[i, j]$.

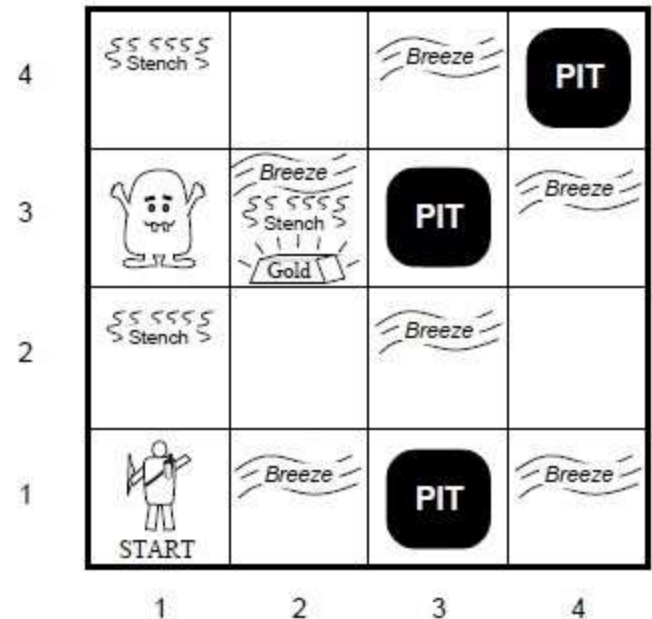
Let $B_{i,j}$ be true if there is a breeze in $[i, j]$.

$\neg P_{1,1}$

$\neg B_{1,1}$

$B_{2,1}$

“Pits cause breezes in adjacent squares”





Wumpus World Sentences

Let $P_{i,j}$ be true if there is a pit in $[i, j]$.

Let $B_{i,j}$ be true if there is a breeze in $[i, j]$.

$$\neg P_{1,1} \quad R_1$$

$$\neg B_{1,1} \quad R_4$$

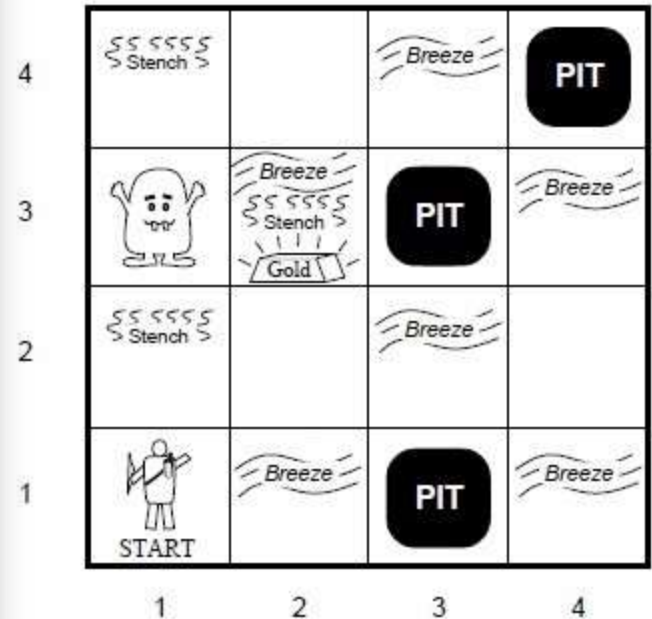
$$B_{2,1} \quad R_5$$

"Pits cause breezes in adjacent squares"

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}) \quad R_2$$

$$B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1}) \quad R_3$$

"A square is breezy **if and only if** there is an adjacent pit"



Wumpus World Truth Table

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	false	false	false	false	false	false	true	true	true	true	false	false
false	false	false	false	false	false	true	true	true	false	true	false	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	true	false	false	false	false	false	true	true	false	true	true	false
false	true	false	false	false	false	true	true	true	true	true	true	<u>true</u>
false	true	false	false	false	true	false	true	true	true	true	true	<u>true</u>
false	true	false	false	false	true	true	true	true	true	true	true	<u>true</u>
false	true	false	false	true	false	false	true	false	false	true	true	false
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	true	true	true	true	true	true	false	true	true	false	true	false

Enumerate rows (different assignments to symbols),

if KB is true in row, check that α is too

128 possible rows, only 3 make the KB true

$P_{1,2}$ is false for all 3; hence, no pit in $P_{1,2}$

Evaluate the entailed sentences in these rows

$P_{2,2}$ is both false and true; hence, no conclusion can be drawn about the pit being in $P_{2,2}$



Inference by Enumeration

Depth-first enumeration of all models is sound and complete

function **TT-ENTAILS?**(KB, α) **returns** *true* or *false*

inputs: KB , the knowledge base, a sentence in propositional logic
 α , the query, a sentence in propositional logic

$symbols \leftarrow$ a list of the proposition symbols in KB and α

return **TT-CHECK-ALL**($KB, \alpha, symbols, []$)

function **TT-CHECK-ALL**($KB, \alpha, symbols, model$) **returns** *true* or *false*

if **EMPTY?**($symbols$) **then**

if **PL-TRUE?**($KB, model$) **then return** **PL-TRUE?**($\alpha, model$)

else return *true*

else do

$P \leftarrow \text{FIRST}(symbols); rest \leftarrow \text{REST}(symbols)$

return **TT-CHECK-ALL**($KB, \alpha, rest, \text{EXTEND}(P, true, model)$) **and**

TT-CHECK-ALL($KB, \alpha, rest, \text{EXTEND}(P, false, model)$)

$O(2^n)$ for n symbols; problem is **co-NP-complete**



Logical Equivalence

Two sentences are logically equivalent iff true in same models:

$\alpha \equiv \beta$ if and only if $\alpha \models \beta$ and $\beta \models \alpha$

$$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha) \quad \text{commutativity of } \wedge$$

$$(\alpha \vee \beta) \equiv (\beta \vee \alpha) \quad \text{commutativity of } \vee$$

$$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma)) \quad \text{associativity of } \wedge$$

$$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma)) \quad \text{associativity of } \vee$$

$$\neg(\neg\alpha) \equiv \alpha \quad \text{double-negation elimination}$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha) \quad \text{contraposition}$$

$$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta) \quad \text{implication elimination}$$

$$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) \quad \text{biconditional elimination}$$

$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \quad \text{De Morgan}$$

$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \quad \text{De Morgan}$$

$$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) \quad \text{distributivity of } \wedge \text{ over } \vee$$

$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) \quad \text{distributivity of } \vee \text{ over } \wedge$$



Validity and Satisfiability

A sentence is **valid** if it is true in **all** models,

e.g., *True*, $A \vee \neg A$, $A \Rightarrow A$, $(A \wedge (A \Rightarrow B)) \Rightarrow B$

Validity is connected to inference via the Deduction Theorem:

$KB \models \alpha$ if and only if $(KB \Rightarrow \alpha)$ is valid $\longrightarrow$ **If KB is true, alpha is always true.** Hence, I can say that alpha follows from KB

A sentence is **satisfiable** if it is true in **some** model

e.g., $A \vee B$, C

A sentence is **unsatisfiable** if it is true in **no** models

e.g., $A \wedge \neg A$

Satisfiability is connected to inference via the following:

$KB \models \alpha$ if and only if $(KB \wedge \neg \alpha)$ is unsatisfiable
i.e., prove α by *reductio ad absurdum*

Suppose that $KB = \text{true}$. Then, this will become unsatisfiable only when alpha is true. Hence, I can say that alpha follows from KB



Two Famous Inference Rules

- Modus Ponens

$$\frac{P \Rightarrow Q, P}{Q}$$

Given that P implies Q,
and I know that P is
true, then I can infer Q

- And Rule

$$P \wedge Q \longrightarrow$$

Given that P AND Q is
true, I can infer that P is
true, and I can also infer
that Q is true.

Resolution

Conjunctive Normal Form (CNF—universal)
conjunction of **disjunctions** of **literals**
clauses

E.g., $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$

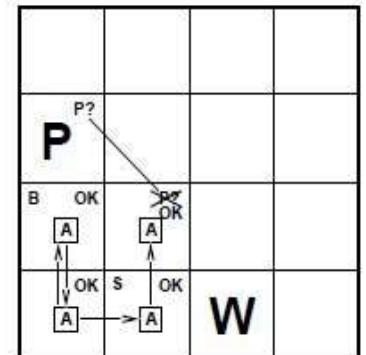
Resolution inference rule (for CNF): complete for propositional logic

$$\frac{l_1 \vee \dots \vee l_k, \quad m_1 \vee \dots \vee m_n}{l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n}$$

where l_i and m_j are complementary literals. E.g.,

$$\frac{P_{1,3} \vee P_{2,2}, \quad \neg P_{2,2}}{P_{1,3}}$$

Resolution is sound and complete for propositional logic





Conversion to CNF

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

1. Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

2. Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$.

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

3. Move $\neg$ inwards using de Morgan's rules and double-negation:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$$

4. Apply distributivity law ($\vee$ over $\wedge$) and flatten:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$



Resolution Algorithm

- Resolution rule:

$$\begin{array}{l} \alpha \vee \beta \\ \neg\beta \vee \gamma \\ \hline \alpha \vee \gamma \end{array}$$

- Resolution refutation:
 - Convert all sentences to CNF
 - Negate the desired conclusion (converted to CNF)
 - Apply resolution rule until either
 - Derive false (a contradiction)
 - Can't apply any more
- Resolution refutation is sound and complete
 - If we derive a contradiction, then the conclusion follows from the axioms
 - If we can't apply any more, then the conclusion cannot be proved from the axioms.

Example

Propositional Resolution Example

Prove R

1	$P \vee Q$
2	$P \rightarrow R$
3	$Q \rightarrow R$

false $\vee$ R

$\neg R \vee$ false

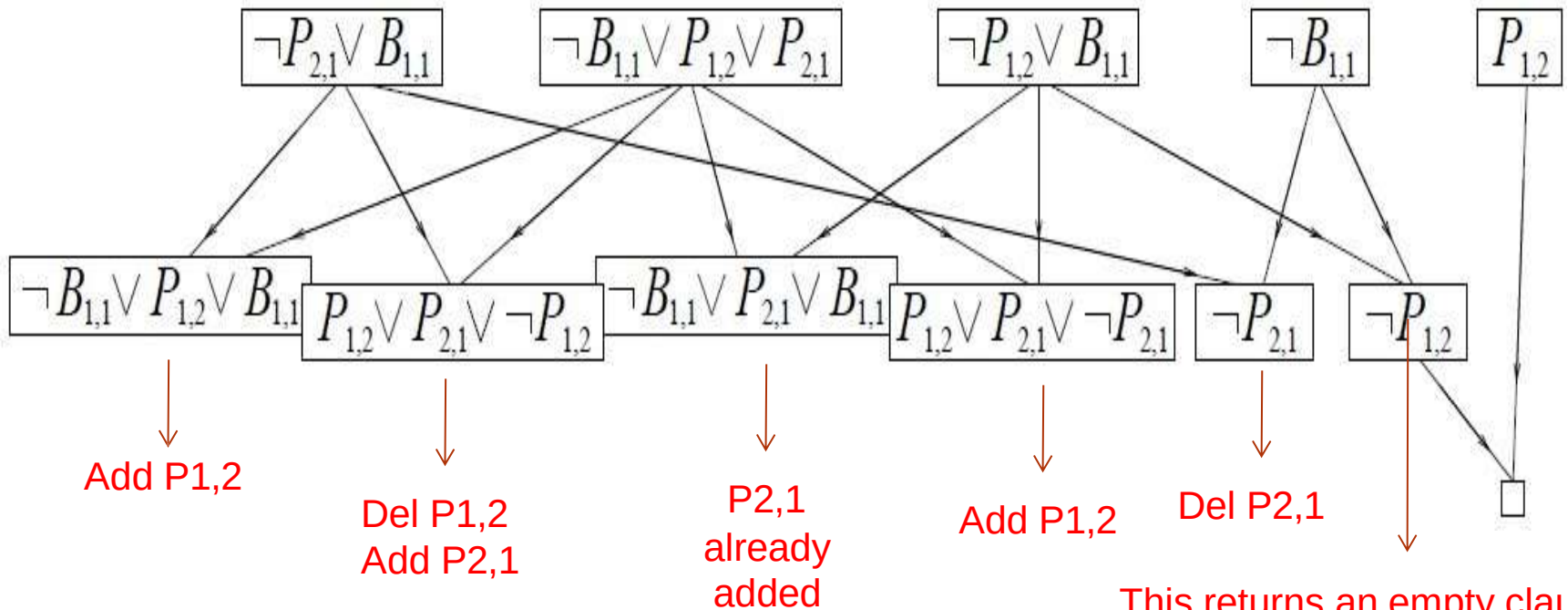
false $\vee$ false

Step	Formula	Derivation
1	$P \vee Q$	Given
2	$\neg P \vee R$	Given
3	$\neg Q \vee R$	Given
4	$\neg R$	Negated conclusion
5	$Q \vee R$	1,2
6	$\neg P$	2,4
7	$\neg Q$	3,4
8	R	5,7
9	•	4,8

Resolution Example

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

$$KB = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1} \quad \alpha = \neg P_{1,2}$$





Resolution Algorithm

function PL-RESOLUTION(KB, α) **returns** *true* or *false*

inputs: KB , the knowledge base, a sentence in propositional logic
 α , the query, a sentence in propositional logic

$clauses \leftarrow$ the set of clauses in the CNF representation of $KB \wedge \neg\alpha$

$new \leftarrow \{ \}$

loop do

for each pair of clauses C_i, C_j **in** $clauses$ **do**

$resolvents \leftarrow$ PL-RESOLVE(C_i, C_j)

if $resolvents$ contains the empty clause **then return** *true*

$new \leftarrow new \cup resolvents$

if $new \subseteq clauses$ **then return** *false*

$clauses \leftarrow clauses \cup new$



Pros and Cons of Propositional Logic

- 😊 Propositional logic is **declarative**: pieces of syntax correspond to facts
- 😊 Propositional logic allows partial/disjunctive/negated information (unlike most data structures and databases)
- 😊 Propositional logic is **compositional**:
meaning of $B_{1,1} \wedge P_{1,2}$ is derived from meaning of $B_{1,1}$ and of $P_{1,2}$
- 😊 Meaning in propositional logic is **context-independent**
(unlike natural language, where meaning depends on context)
- 😞 Propositional logic has very limited expressive power
(unlike natural language)
E.g., cannot say “pits cause breezes in adjacent squares”
except by writing one sentence for each square



First-Order Logic

Whereas propositional logic assumes world contains **facts**,
first-order logic (like natural language) assumes the world contains

- **Objects:** people, houses, numbers, theories, Ronald McDonald, colors, baseball games, wars, centuries ...
- **Relations:** red, round, bogus, prime, multistoried ...,
brother of, bigger than, inside, part of, has color, occurred after, owns,
comes between, ...
- **Functions:** father of, best friend, third inning of, one more than, end of
...

Constants	<i>KingJohn, 2, UCB, ...</i>
Predicates	<i>Brother, >, ...</i>
Functions	<i>Sqrt, LeftLegOf, ...</i>
Syntax: Variables	<i>x, y, a, b, ...</i>
Connectives	$\wedge \vee \neg \Rightarrow \Leftrightarrow$
Equality	$=$
Quantifiers	$\forall \exists$



Atomic Sentences

Atomic sentence = $\text{predicate}(\text{term}_1, \dots, \text{term}_n)$
or $\text{term}_1 = \text{term}_2$

Term = $\text{function}(\text{term}_1, \dots, \text{term}_n)$
or *constant* or *variable*

E.g., $\text{Brother}(\text{KingJohn}, \text{RichardTheLionheart})$
 $> (\text{Length}(\text{LeftLegOf}(\text{Richard})), \text{Length}(\text{LeftLegOf}(\text{KingJohn})))$



Complex Sentences

Complex sentences are made from atomic sentences using connectives

$$\neg S, \quad S_1 \wedge S_2, \quad S_1 \vee S_2, \quad S_1 \Rightarrow S_2, \quad S_1 \Leftrightarrow S_2$$

E.g. $Sibling(KingJohn, Richard) \Rightarrow Sibling(Richard, KingJohn)$
 $>(1, 2) \vee \leq(1, 2)$
 $>(1, 2) \wedge \neg >(1, 2)$



Truth in FOL

Sentences are true with respect to a model and an interpretation

Model contains ≥ 1 objects (domain elements) and relations among them

Interpretation specifies referents for

constant symbols $\rightarrow$ objects

predicate symbols $\rightarrow$ relations

function symbols $\rightarrow$ functional relations

An atomic sentence $predicate(term_1, \dots, term_n)$ is true
iff the objects referred to by $term_1, \dots, term_n$
are in the relation referred to by $predicate$





Truth Example

Consider the interpretation in which

Richard → Richard the Lionheart

John → the evil King John

Brother → the brotherhood relation

Under this interpretation, *Brother(Richard, John)* is true just in case Richard the Lionheart and the evil King John are in the brotherhood relation in the model





Universal Quantification

$\forall \langle \text{variables} \rangle \langle \text{sentence} \rangle$

Everyone at Berkeley is smart:

$\forall x \text{ At}(x, \text{Berkeley}) \Rightarrow \text{Smart}(x)$

$\forall x P$ is true in a model m iff P is true with x being **each** possible object in the model

Roughly speaking, equivalent to the conjunction of instantiations of P

$(\text{At}(\text{KingJohn}, \text{Berkeley}) \Rightarrow \text{Smart}(\text{KingJohn}))$
 $\wedge (\text{At}(\text{Richard}, \text{Berkeley}) \Rightarrow \text{Smart}(\text{Richard}))$
 $\wedge (\text{At}(\text{Berkeley}, \text{Berkeley}) \Rightarrow \text{Smart}(\text{Berkeley}))$
 $\wedge \dots$

A Common Mistake To Avoid:

Typically, $\Rightarrow$ is the main connective with $\forall$

Common mistake: using $\wedge$ as the main connective with $\forall$:

$\forall x \text{ At}(x, \text{Berkeley}) \wedge \text{Smart}(x)$

means “Everyone is at Berkeley and everyone is smart”



Existential Quantification

$\exists \langle \text{variables} \rangle \langle \text{sentence} \rangle$

Someone at Stanford is smart:

$\exists x \text{ At}(x, \text{Stanford}) \wedge \text{Smart}(x)$

$\exists x P$ is true in a model m iff P is true with x being **some** possible object in the model

Roughly speaking, equivalent to the disjunction of instantiations of P

$(\text{At}(\text{KingJohn}, \text{Stanford}) \wedge \text{Smart}(\text{KingJohn}))$
 $\vee (\text{At}(\text{Richard}, \text{Stanford}) \wedge \text{Smart}(\text{Richard}))$
 $\vee (\text{At}(\text{Stanford}, \text{Stanford}) \wedge \text{Smart}(\text{Stanford}))$
 $\vee \dots$

Another Common Mistake To Avoid:

Typically, $\wedge$ is the main connective with $\exists$

Common mistake: using $\Rightarrow$ as the main connective with $\exists$:

$\exists x \text{ At}(x, \text{Stanford}) \Rightarrow \text{Smart}(x)$



Properties of Quantifiers

$\forall x \forall y$ is the same as $\forall y \forall x$

$\exists x \exists y$ is the same as $\exists y \exists x$

$\exists x \forall y$ is **not** the same as $\forall y \exists x$

$\exists x \forall y \text{ Loves}(x, y)$

“There is a person who loves everyone in the world”

$\forall y \exists x \text{ Loves}(x, y)$

“Everyone in the world is loved by at least one person”

Quantifier duality: each can be expressed using the other

$\forall x \text{ Likes}(x, \text{IceCream}) \quad \neg \exists x \neg \text{Likes}(x, \text{IceCream})$

$\exists x \text{ Likes}(x, \text{Broccoli}) \quad \neg \forall x \neg \text{Likes}(x, \text{Broccoli})$



Fun With Sentences

Brothers are siblings

$$\forall x, y \text{ Brother}(x, y) \Rightarrow \text{Sibling}(x, y).$$

“Sibling” is symmetric

$$\forall x, y \text{ Sibling}(x, y) \Leftrightarrow \text{Sibling}(y, x).$$

One's mother is one's female parent

$$\forall x, y \text{ Mother}(x, y) \Leftrightarrow (\text{Female}(x) \wedge \text{Parent}(x, y)).$$

A first cousin is a child of a parent's sibling

$$\forall x, y \text{ FirstCousin}(x, y) \Leftrightarrow \exists p, ps \text{ Parent}(p, x) \wedge \text{Sibling}(ps, p) \wedge \text{Parent}(ps, y)$$

Equality:

$term_1 = term_2$ is true under a given interpretation
if and only if $term_1$ and $term_2$ refer to the same object

E.g., definition of (full) *Sibling* in terms of *Parent*:

$$\forall x, y \text{ Sibling}(x, y) \Leftrightarrow [\neg(x = y) \wedge \exists m, f \neg(m = f) \wedge \text{Parent}(m, x) \wedge \text{Parent}(f, x) \wedge \text{Parent}(m, y) \wedge \text{Parent}(f, y)]$$



Deducing Hidden Properties

Properties of locations:

$$\forall x, t \text{ } At(Agent, x, t) \wedge Smelt(t) \Rightarrow Smelly(x)$$

$$\forall x, t \text{ } At(Agent, x, t) \wedge Breeze(t) \Rightarrow Breezy(x)$$

Squares are breezy near a pit:

Diagnostic rule—infer cause from effect

$$\forall y \text{ } Breezy(y) \Rightarrow \exists x \text{ } Pit(x) \wedge Adjacent(x, y)$$

Causal rule—infer effect from cause

$$\forall x, y \text{ } Pit(x) \wedge Adjacent(x, y) \Rightarrow Breezy(y)$$

Neither of these is complete—e.g., the causal rule doesn't say whether squares far away from pits can be breezy

Definition for the *Breezy* predicate:

$$\forall y \text{ } Breezy(y) \Leftrightarrow [\exists x \text{ } Pit(x) \wedge Adjacent(x, y)]$$



Universal Instantiation

Every instantiation of a universally quantified sentence is entailed by it:

$$\frac{\forall v \alpha}{\text{SUBST}(\{v/g\}, \alpha)}$$

Stands for substitution

for any variable v and ground term g

E.g., $\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$ yields

Typically a constant, which substitutes the variable

$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$

$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$

$\text{King}(\text{Father}(\text{John})) \wedge \text{Greedy}(\text{Father}(\text{John})) \Rightarrow \text{Evil}(\text{Father}(\text{John}))$

$\vdots$

Once the substitution is made, we can entail the new sentence



Existential Instantiation

For any sentence α , variable v , and constant symbol k that does not appear elsewhere in the knowledge base:

$$\frac{\exists v \alpha}{\text{SUBST}(\{v/k\}, \alpha)}$$

E.g., $\exists x \text{Crown}(x) \wedge \text{OnHead}(x, \text{John})$ yields

$$\text{Crown}(C_1) \wedge \text{OnHead}(C_1, \text{John})$$

provided C_1 is a new constant symbol, called a Skolem constant

Another example: from $\exists x \, d(x^y)/dy = x^y$ we obtain

$$d(e^y)/dy = e^y$$

provided e is a new constant symbol

It should not appear elsewhere in the data base, because of the existential quantifier (i.e., there exists....). So we assume the minimum value, i.e., there exists just one...



Some Facts

- UI can be applied repeatedly to the same FOL sentence, in order to add new sentences
 - The new KB always remains logically equivalent to the old one
- EI can be applied only once; and once it is applied, then the existentially quantified sentence should be removed from the KB
 - The new KB is not logically equivalent to the old, but rather it is inferentially equivalent (You replace the existentially quantified sentence with an entailed one).



Reduction to Propositional Inference

Suppose the KB contains just the following:

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$

$\text{King}(\text{John})$

$\text{Greedy}(\text{John})$

$\text{Brother}(\text{Richard}, \text{John})$

Instantiating the universal sentence in **all possible** ways, we have

$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$

$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$

$\text{King}(\text{John})$

$\text{Greedy}(\text{John})$

$\text{Brother}(\text{Richard}, \text{John})$

The new KB is propositionalized: proposition symbols are

$\text{King}(\text{John}), \text{Greedy}(\text{John}), \text{Evil}(\text{John}), \text{King}(\text{Richard})$ etc.

↓
For convenience, each one of these can be replaced by one symbol, e.g., A, B, C etc.



Problems of Propositionalization

Propositionalization seems to generate lots of irrelevant sentences.

E.g., from

$$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$$

King(John)

$$\forall y \text{ Greedy}(y)$$

Brother(Richard, John)

it seems obvious that *Evil(John)*, but propositionalization produces lots of facts such as *Greedy(Richard)* that are irrelevant

With p k -ary predicates and n constants, there are $p \cdot n^k$ instantiations

With function symbols, it gets much much worse!



Unification

We can get the inference immediately if we can find a substitution θ such that $King(x)$ and $Greedy(x)$ match $King(John)$ and $Greedy(y)$

$\theta = \{x/John, y/John\}$ works

$UNIFY(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$

Unification is all about **finding substitutions** in order to make two expressions equal

p	q	θ
$Knows(John, x)$	$Knows(John, Jane)$	
$Knows(John, x)$	$Knows(y, OJ)$	
$Knows(John, x)$	$Knows(y, Mother(y))$	
$Knows(John, x)$	$Knows(x, OJ)$	

What substitution is required in order to make these two expressions equal?



Unification

We can get the inference immediately if we can find a substitution θ such that $King(x)$ and $Greedy(x)$ match $King(John)$ and $Greedy(y)$

$\theta = \{x/John, y/John\}$ works

$UNIFY(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$

p	q	θ
$Knows(John, x)$	$Knows(John, Jane)$	$\{x/Jane\}$
$Knows(John, x)$	$Knows(y, OJ)$	
$Knows(John, x)$	$Knows(y, Mother(y))$	
$Knows(John, x)$	$Knows(x, OJ)$	



Unification

We can get the inference immediately if we can find a substitution θ such that $King(x)$ and $Greedy(x)$ match $King(John)$ and $Greedy(y)$

$\theta = \{x/John, y/John\}$ works

$UNIFY(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$

p	q	θ
$Knows(John, x)$	$Knows(John, Jane)$	$\{x/Jane\}$
$Knows(John, x)$	$Knows(y, OJ)$	$\{x/OJ, y/John\}$
$Knows(John, x)$	$Knows(y, Mother(y))$	
$Knows(John, x)$	$Knows(x, OJ)$	



Unification

We can get the inference immediately if we can find a substitution θ such that $King(x)$ and $Greedy(x)$ match $King(John)$ and $Greedy(y)$

$\theta = \{x/John, y/John\}$ works

$UNIFY(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$

p	q	θ
$Knows(John, x)$	$Knows(John, Jane)$	$\{x/Jane\}$
$Knows(John, x)$	$Knows(y, OJ)$	$\{x/OJ, y/John\}$
$Knows(John, x)$	$Knows(y, Mother(y))$	$\{y/John, x/Mother(John)\}$
$Knows(John, x)$	$Knows(x, OJ)$	



Unification

We can get the inference immediately if we can find a substitution θ such that $King(x)$ and $Greedy(x)$ match $King(John)$ and $Greedy(y)$

$\theta = \{x/John, y/John\}$ works

$UNIFY(\alpha, \beta) = \theta$ if $\alpha\theta = \beta\theta$

p	q	θ
$Knows(John, x)$	$Knows(John, Jane)$	$\{x/Jane\}$
$Knows(John, x)$	$Knows(y, OJ)$	$\{x/OJ, y/John\}$
$Knows(John, x)$	$Knows(y, Mother(y))$	$\{y/John, x/Mother(John)\}$
$Knows(John, x)$	$Knows(x, OJ)$	fail

Standardizing apart eliminates overlap of variables, e.g., $Knows(z_{17}, OJ)$



Generalized Modus Ponens

$$\frac{p_1', p_2', \dots, p_n', (p_1 \wedge p_2 \wedge \dots \wedge p_n \Rightarrow q)}{q\theta}$$

where $p_i'\theta = p_i\theta$ for all i

p_1' is *King(John)* p_1 is *King(x)*
 p_2' is *Greedy(y)* p_2 is *Greedy(x)*
 θ is $\{x/\text{John}, y/\text{John}\}$ q is *Evil(x)*
 $q\theta$ is *Evil(John)*

i.e., with statements
of this format

GMP used with KB of definite clauses (**exactly** one positive literal)

All variables assumed universally quantified

Something important to
remember



Example Knowledge Base (EKB)

- The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American.
- Prove that Col. West is a criminal





EKB

... it is a crime for an American to sell weapons to hostile nations:

$$\textit{American}(x) \wedge \textit{Weapon}(y) \wedge \textit{Sells}(x, y, z) \wedge \textit{Hostile}(z) \Rightarrow \textit{Criminal}(x)$$

Nono ... has some missiles

$$\textit{Owns}(\textit{Nono}, M_1) \text{ and } \textit{Missile}(M_1)$$

... all of its missiles were sold to it by Colonel West

$$\forall x \textit{ Missile}(x) \wedge \textit{Owns}(\textit{Nono}, x) \Rightarrow \textit{Sells}(\textit{West}, x, \textit{Nono})$$

Missiles are weapons:

$$\textit{Missile}(x) \Rightarrow \textit{Weapon}(x)$$



EKB

An enemy of America counts as “hostile”:

$$\textit{Enemy}(x, \textit{America}) \Rightarrow \textit{Hostile}(x)$$

West, who is American ...

$$\textit{American}(\textit{West})$$

The country Nono, an enemy of America ...

$$\textit{Enemy}(\textit{Nono}, \textit{America})$$





Forward Chaining Algorithm

function **FOL-FC-Ask**(KB, α) returns a substitution or *false*

repeat until *new* is empty

$new \leftarrow \{ \}$

for each sentence r in KB do

$(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE-APART}(r)$

for each θ such that $(p_1 \wedge \dots \wedge p_n)\theta = (p'_1 \wedge \dots \wedge p'_n)\theta$

for some $p'_1, \dots, p'_n$ in KB

$q' \leftarrow \text{SUBST}(\theta, q)$

if q' is not a renaming of a sentence already in KB or *new* then do

add q' to *new*

$\phi \leftarrow \text{UNIFY}(q', \alpha)$

if ϕ is not *fail* then return ϕ

add *new* to KB

return *false*



Forward Chaining Proof

All the basic facts that have been
derived



American(West)

Missile(M1)

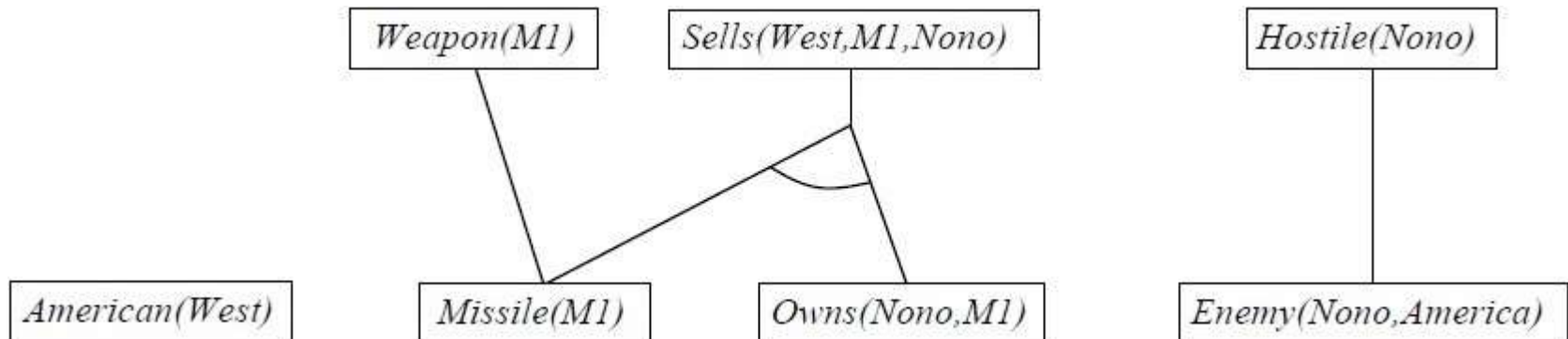
Owns(Nono,M1)

Enemy(Nono,America)

Start from the basic facts at the bottom. Then, you go through a series of iterations; each iteration is represented by trying to go one level upward from the current one. In each iteration, we write what we can infer (using unification on implication sentences only) from the bottom level. Basically, we can infer the consequent if the premise is satisfied with some substitution.



Forward Chaining Proof



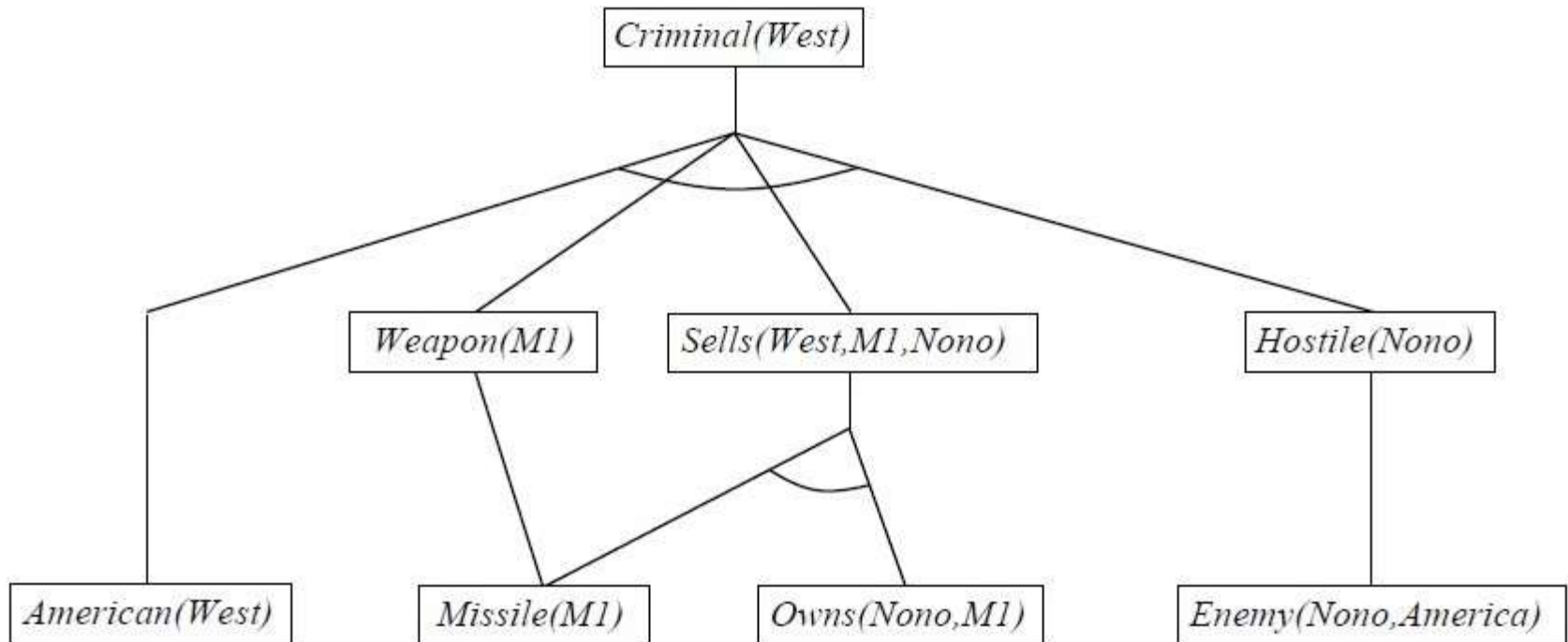
First Iteration (unification and reasoning is possible only on the following three implications)

$Missile(x) \Rightarrow Weapon(x)$ **Substitution: $\{x|M1\}$**

$\forall x \text{ Missile}(x) \wedge Owns(Nono, x) \Rightarrow Sells(West, x, Nono)$ **Substitution: $\{x|M1\}$**

$Enemy(x, America) \Rightarrow Hostile(x)$ **Substitution: $\{x|Nono\}$**

Forward Chaining Proof



Second Iteration: Only one implication is now possible on the following rule with substitution $\{x|West, y|M1, z|Nono\}$

$$American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x)$$



Backward Chaining Algorithm

function **FOL-BC-ASK**($KB, goals, \theta$) returns a set of substitutions

inputs: KB , a knowledge base

$goals$, a list of conjuncts forming a query (θ already applied)

θ , the current substitution, initially the empty substitution $\{ \}$

local variables: $answers$, a set of substitutions, initially empty

if $goals$ is empty then return $\{ \theta \}$

$q' \leftarrow \text{SUBST}(\theta, \text{FIRST}(goals))$

for each sentence r in KB

where $\text{STANDARDIZE-APART}(r) = (p_1 \wedge \dots \wedge p_n \Rightarrow q)$

and $\theta' \leftarrow \text{UNIFY}(q, q')$ succeeds

$new_goals \leftarrow [p_1, \dots, p_n | \text{REST}(goals)]$

$answers \leftarrow \text{FOL-BC-ASK}(KB, new_goals, \text{COMPOSE}(\theta', \theta)) \cup answers$

return $answers$



Backward Chaining Example

Criminal(West) $\longrightarrow$ Goal/query

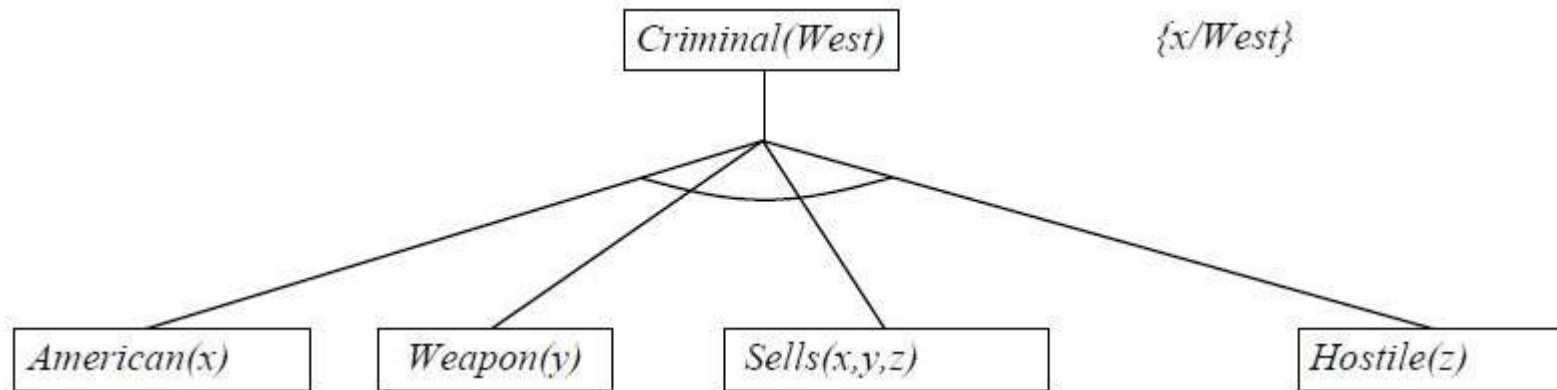
Work backward from the goal (query), chaining through implications in order to find facts that support the goal.

The algorithm returns a set of substitutions that satisfy the goal

It simply considers a goal, and finds every clause in the knowledge base whose positive literal (consequent) satisfies with this goal

When this condition is satisfied, a new recursive call is generated in which the antecedent of the rule is added at the next (bottom) level

Backward Chaining Example

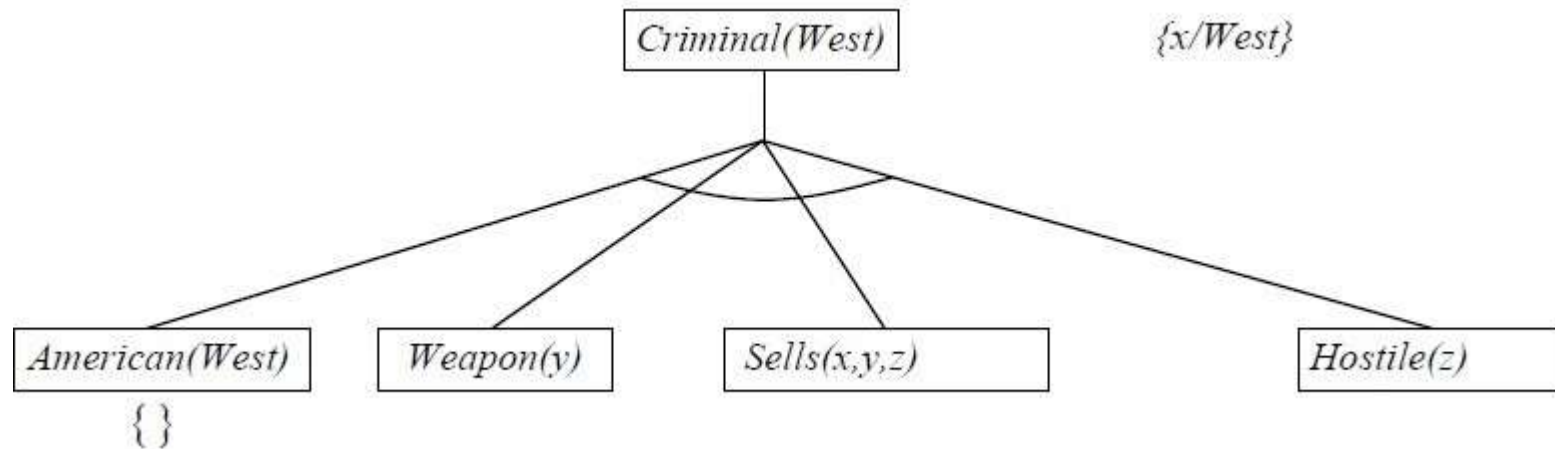


Criminal(West) can be unified with Criminal(x) with the substitution $\{x/West\}$:

$$American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x)$$

We first generate the literals in the antecedent

Backward Chaining Example

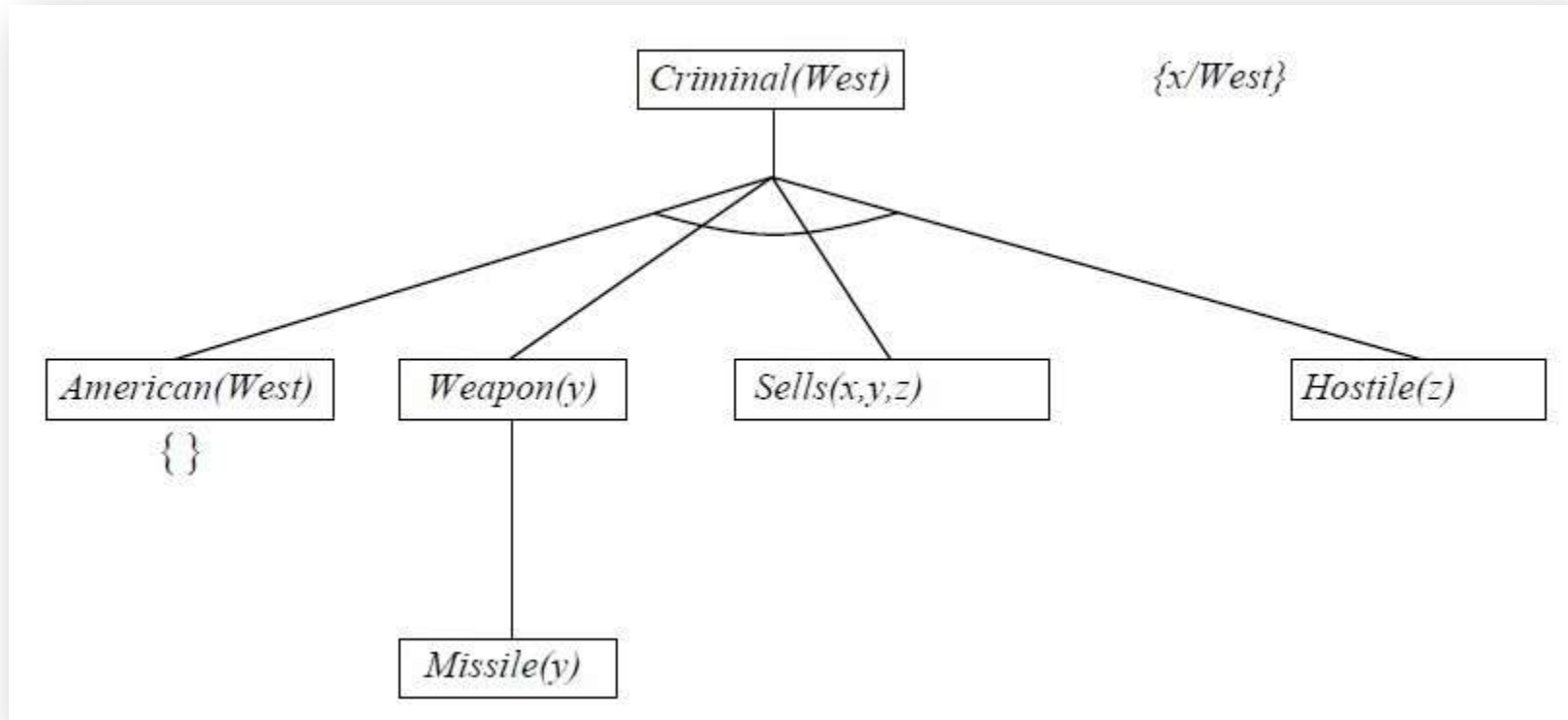


Criminal(West) can be unified with Criminal(x) with the substitution $\{x/West\}$:

$$American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x)$$

Then, we move depth-first through the literals, making the substitution $\{x/West\}$

Backward Chaining Example

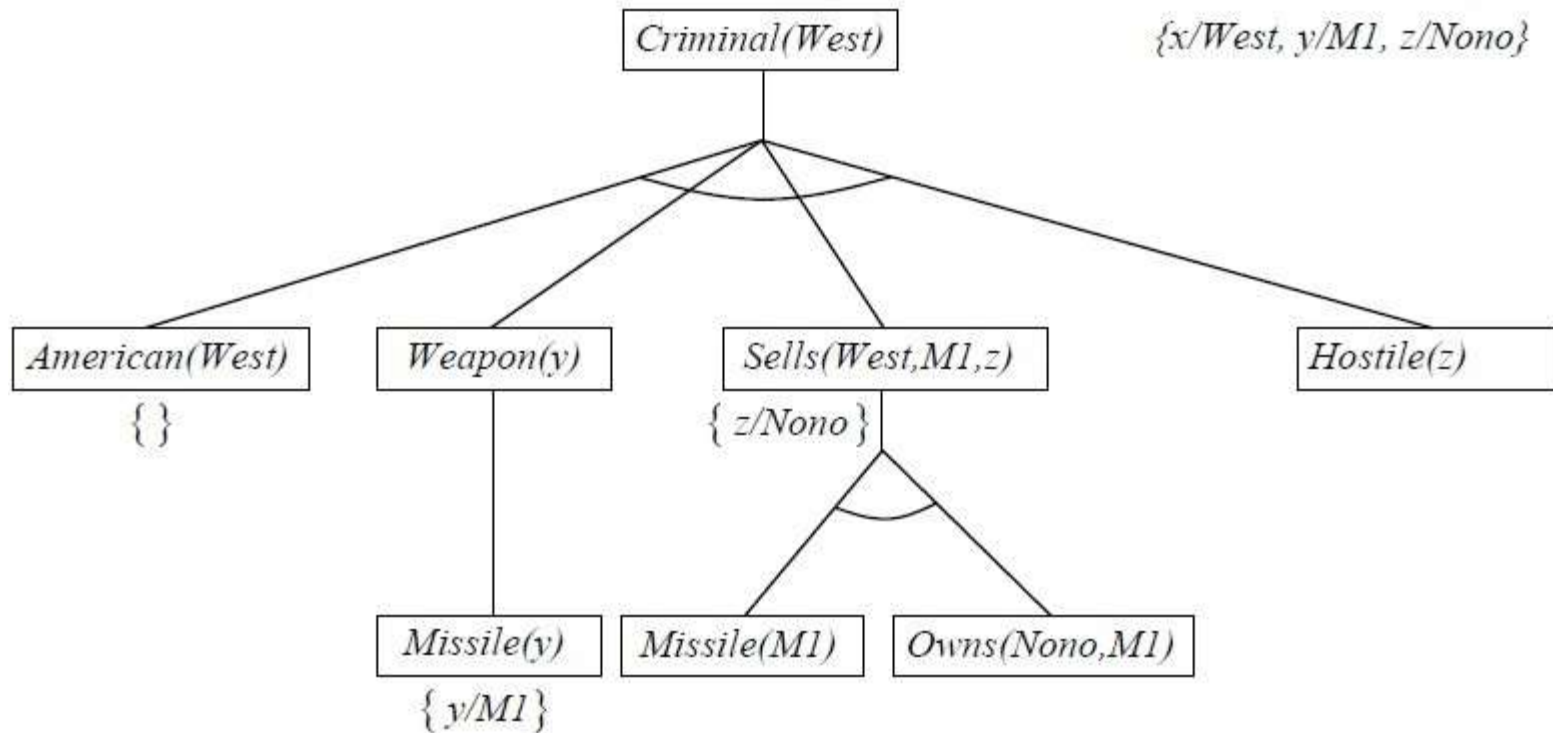


Weapon(y) can be unified with the consequent Weapon(x): $Missile(x) \Rightarrow Weapon(x)$

**The difference in variables doesn't matter; the concept is the same,
i.e., x or y is a weapon**

So we generate its antecedent, i.e., Missile(y)

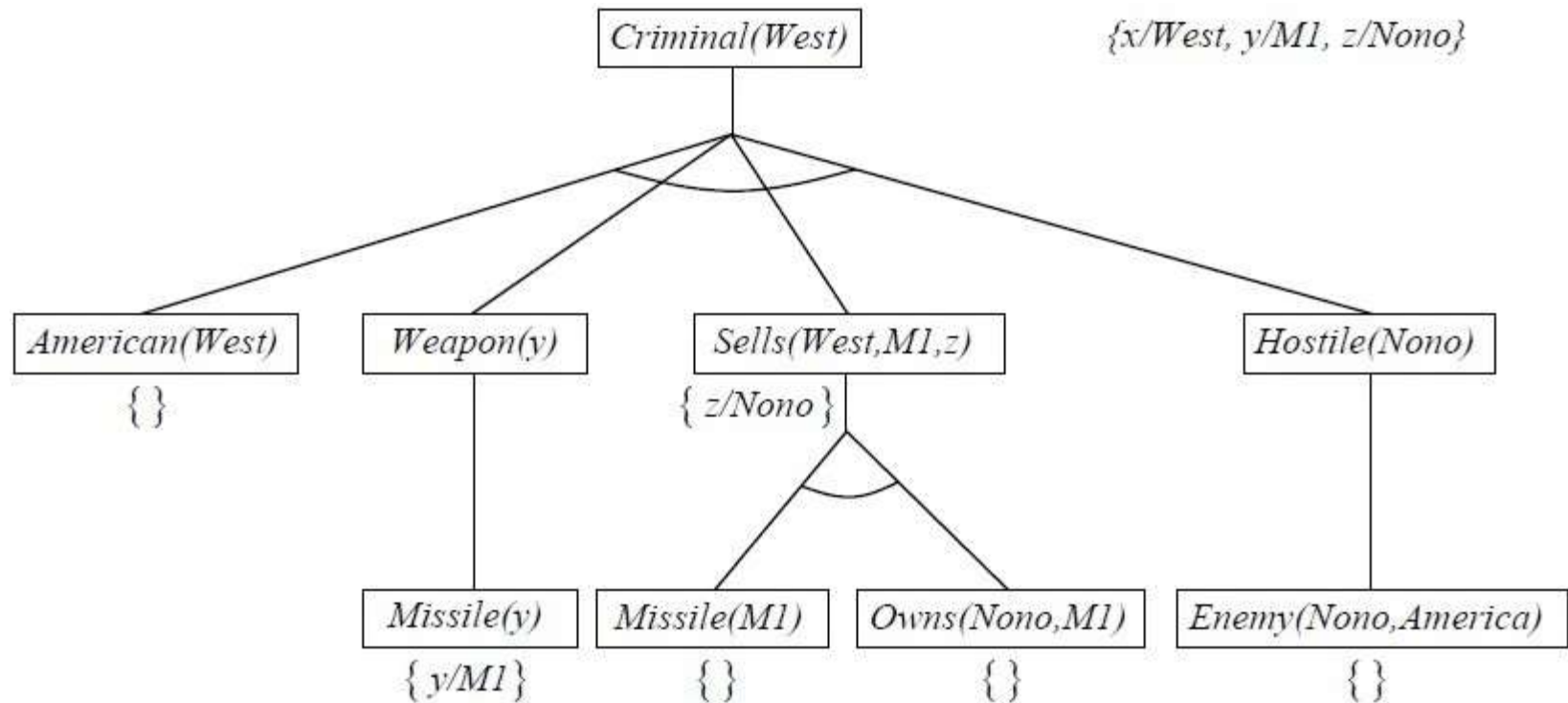
Backward Chaining Example



Missile(M1) unifies with Missile(y) with {y|M1}

Now, generate antecedents for Sells, and assign {z|Nono}

Backward Chaining Example



With $\{z/Nono\}$, we get $Hostile(Nono)$

Which unifies with $Hostile(x)$ $Enemy(x, America) \Rightarrow Hostile(x)$



Properties of Backward Chaining

Depth-first recursive proof search: space is linear in size of proof

Incomplete due to infinite loops

⇒ fix by checking current goal against every goal on stack

Inefficient due to repeated subgoals (both success and failure)

⇒ fix using caching of previous results (extra space!)

Widely used (without improvements!) for logic programming





Resolution: A Brief Summary

Full first-order version:

$$\frac{\ell_1 \vee \dots \vee \ell_k, \quad m_1 \vee \dots \vee m_n}{(\ell_1 \vee \dots \vee \ell_{i-1} \vee \ell_{i+1} \vee \dots \vee \ell_k \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n)\theta}$$

where $\text{UNIFY}(\ell_i, \neg m_j) = \theta$.

For example,

$$\frac{\neg \text{Rich}(x) \vee \text{Unhappy}(x) \quad \text{Rich}(\text{Ken})}{\text{Unhappy}(\text{Ken})}$$

with $\theta = \{x/\text{Ken}\}$

Apply resolution steps to $\text{CNF}(KB \wedge \neg \alpha)$; complete for FOL



Conversion to CNF

Everyone who loves all animals is loved by someone:

$$\forall x [\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y)] \Rightarrow [\exists y \text{ Loves}(y, x)]$$

1. Eliminate biconditionals and implications

$$\forall x [\neg \forall y \neg \text{Animal}(y) \vee \text{Loves}(x, y)] \vee [\exists y \text{ Loves}(y, x)]$$

2. Move $\neg$ inwards: $\neg \forall x, p \equiv \exists x \neg p$, $\neg \exists x, p \equiv \forall x \neg p$:

$$\forall x [\exists y \neg(\neg \text{Animal}(y) \vee \text{Loves}(x, y))] \vee [\exists y \text{ Loves}(y, x)]$$

$$\forall x [\exists y \neg \neg \text{Animal}(y) \wedge \neg \text{Loves}(x, y)] \vee [\exists y \text{ Loves}(y, x)]$$

$$\forall x [\exists y \text{ Animal}(y) \wedge \neg \text{Loves}(x, y)] \vee [\exists y \text{ Loves}(y, x)]$$



Conversion to CNF

3. Standardize variables: each quantifier should use a different one

$$\forall x [\exists y \text{ Animal}(y) \wedge \neg \text{Loves}(x, y)] \vee [\exists z \text{ Loves}(z, x)]$$

4. Skolemize: a more general form of existential instantiation.
Each existential variable is replaced by a Skolem function of the enclosing universally quantified variables:

$$\forall x [\text{Animal}(F(x)) \wedge \neg \text{Loves}(x, F(x))] \vee \text{Loves}(G(x), x)$$

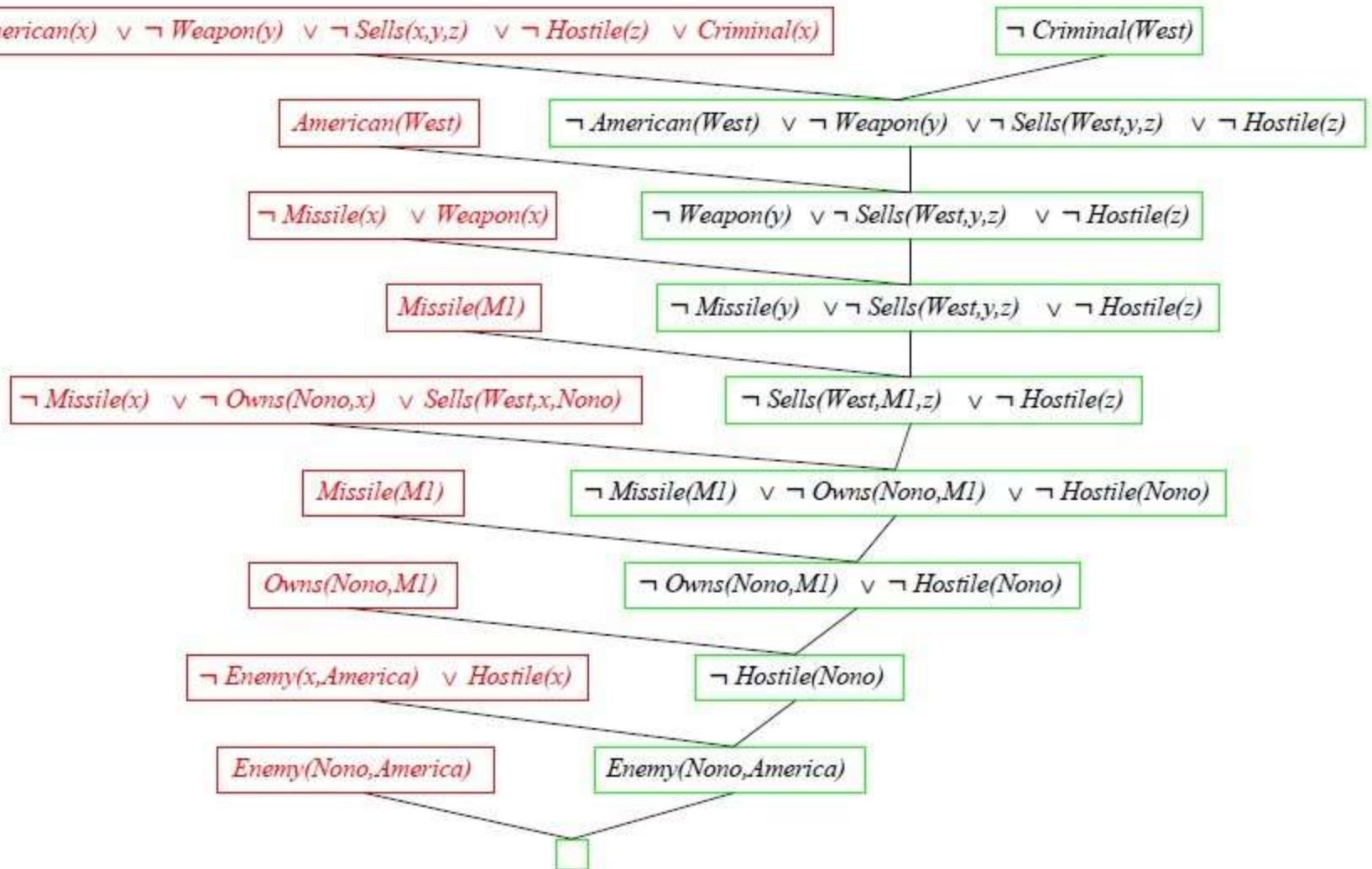
5. Drop universal quantifiers:

$$[\text{Animal}(F(x)) \wedge \neg \text{Loves}(x, F(x))] \vee \text{Loves}(G(x), x)$$

6. Distribute $\wedge$ over $\vee$:

$$[\text{Animal}(F(x)) \vee \text{Loves}(G(x), x)] \wedge [\neg \text{Loves}(x, F(x)) \vee \text{Loves}(G(x), x)]$$

Resolution Proof: Definite Clauses





Resolution Proof

- In the previous slide, the squares marked in red are nothing but the consecutive goals in the backward chaining procedure.
- In fact, backward chaining is really just a special case of resolution, with a particular control strategy to decide which resolution to perform next.

